Universidade Estadual da Paraíba

ES6

PROFESSOR:

Allan Vilar de Carvalho

allanvilarcarvalho@gmail.com

Tópicos da Aula

- O que é?
- Comparação de escopo de variável
- Array
- Funções
- Atribuição de desestruturação

ES6

UEPB
Universidade
Estadual da Paraíba

- O que é?
  - versão padronizada de JavaScript
  - todos os principais navegadores seguem
  - ECMAScript x JavaScript x ES
  - ES5: finalizado em 2009
  - ES6: lançado em 2015
    - novos recursos: funções de seta, desestruturação, classes, promessas e módulos

ES6

- Comparação de escopo de variável
  - let e var

  - var
    - Declaração:
      - globalmente
      - localmente: se for declarada dentro de uma função
  - let
    - Declaração:
      - dentro de um bloco, instrução ou expressão: escopo é limitado a esse bloco, instrução ou expressão

ES6

- Comparação de escopo de variável
  - let e var
  - var
    - Ex.:

```
var numArray = [];
for (var i = 0; i < 3; i++) {
  numArray.push(i);
}
console.log(numArray);
console.log(i);
```

```
var numArray = [];
var i;
for (i = 0; i < 3; i++) {
  numArray.push(i);
}
console.log(numArray);
console.log(i);
```

ES6

- Comparação de escopo de variável
  - let e var

  - var
    - Ex.: problema

```
var printNumTwo;
for (var i = 0; i < 3; i++) {
  if (i === 2) {
    printNumTwo = function() {
      return i;
    };
  }
}
console.log(printNumTwo());
```

ES6

- Comparação de escopo de variável
  - let e var
  - let
    - Ex.:

```
let printNumTwo;
for (let i = 0; i < 3; i++) {
  if (i === 2) {
    printNumTwo = function() {
      return i;
    };
  }
}
console.log(printNumTwo());
console.log(i);
```

ES6

- Array declarado com const
  - Uso do const
    - valor mutável
      - arrays/funções atribuídos a variável const
    - apenas evita: reatribuição do identificador da variável
  - Ex.:

```
const s = [5, 6, 7];
s = [1, 2, 3];
s[2] = 45;
console.log(s);
```

ES6

- Impedir mutação de objetos
  - função Object.freeze
    - Ex.: erro

```
let obj = {
  name:"FreeCodeCamp",
  review:"Awesome"
};
Object.freeze(obj);
obj.review = "bad";
obj.newProp = "Test";
console.log(obj);
```

ES6

- Funções de seta
  - Funções anônimas
    - Ex.:

```
const myFunc = function() {
  const myVar = "value";
  return myVar;
}
```

- Ex. 1:

```
const myFunc = () => {
  const myVar = "value";
  return myVar;
}
```

Ex. 2:

```
const myFunc = () => "value";
```

ES6

- Funções de seta: com parâmetros
  - Ex. 1:

```
const doubler = (item) => item * 2;
doubler(4);
```

  - Ex. 2:

```
const doubler = item => item * 2;
```

  - Ex. 3:

```
const multiplier = (item, multi) => item * multi;
multiplier(4, 2);
```

ES6

- Parâmetro padrão: para função
  - Exibido quando o argumento não é especificado
  - Ex.:

```
const greeting = (name = "Anonymous") => "Hello " + name;

console.log(greeting("John"));
console.log(greeting());
```

ES6

- Parâmetro Rest: para função
  - Permite: criar funções que aceitam um número variável de argumentos
  - Armazenamento: array

  - Ex.:

```
function howMany(...args) {
  return "You have passed " + args.length + " arguments.";
}
console.log(howMany(0, 1, 2));
console.log(howMany("string", null, [1, 2, 3], { }));
```

ES6

- Operador Spread: para avaliar matrizes
  - Permite: expandir arrays e outras expressões em locais onde vários parâmetros ou elementos são esperados
  - Ex.:

```
const arr = [6, 89, 3, 45];
const maximus = Math.max(...arr);
```

  - Ex.: ...arr retorna um array descompactado

```
const spreaded = [...arr];
```

ES6

- Atribuição de desestruturação: para extrair valores de objetos

```
const user = { name: 'John Doe', age: 34 };

const name = user.name;
const age = user.age;
```

- Ex.:

```
const { name, age } = user;
```

ES6

- Atribuição de desestruturação: para atribuir variáveis de objetos
  - Permite: atribuir um novo nome de variável ao extrair valores

```
const user = { name: 'John Doe', age: 34 };
```

  - Ex.:

```
const { name: userName, age: userAge } = user;
```

ES6

- Atribuição de desestruturação: para extrair/atribuir variáveis de objetos aninhados

```
const user = {
  johnDoe: {
    age: 34,
    email: 'johnDoe@freeCodeCamp.com'
  }
};
```

- Ex. 1: `const { johnDoe: { age, email }} = user;`
- Ex. 2: `const { johnDoe: { age: userAge, email: userEmail }} = user;`

ES6

- Atribuição de desestruturação: para atribuir variáveis de matrizes
  - Ex.:

```
const [a, b] = [1, 2, 3, 4, 5, 6];
console.log(a, b);
```

  - Ex.: usando vírgulas para chegar ao índice desejado

```
const [a, b,,, c] = [1, 2, 3, 4, 5, 6];
console.log(a, b, c);
```

ES6

- Desestruturação de array: via elementos com Rest
  - Permite: coletar o restante dos elementos de um array em um array separado

```
const [a, b, ...arr] = [1, 2, 3, 4, 5, 7];
console.log(a, b);
console.log(arr);
```

Dúvidas?